

ÉCOLE MILITAIRE POLYTECHNIQUE

Département d'Informatique

RAPPORT DE PROJET · DATA MINING

Prédiction des recommandations d'achats de vêtements pour femmes

Par apprentissage automatique classique

Audit de qualité des données, analyse exploratoire, tests d'hypothèses, comparaison de six modèles de classification et application web Streamlit d'aide à la décision.

RÉALISÉ PAR

SIDOUS Sarah Aicha

LAMRI Nour EL Israa

MISSOUN Bouchra

GHANFOUD Hadil

ENCADRANT

Cdt HOSNI

MODULE

Data Mining et Machine Learning

ANNÉE UNIVERSITAIRE

2026–2027

PROJET

PROJ-RSD18

Python · pandas · scikit-learn · SciPy · Streamlit · Git / GitHub

github.com/RSD018/Prediction-des-recommandations--dachats--de-vetements--pour--femmes

Déclaration du projet

> ***PROJET ACADÉMIQUE** : ce rapport présente un projet réalisé dans le cadre du module de Data Mining. Il utilise exclusivement des algorithmes classiques de classification supervisée, sans apprentissage profond.*

Ce rapport a été rédigé dans le cadre du module de Data Mining. Le code source, les notebooks, les modèles entraînés et l'application sont archivés dans le dépôt GitHub indiqué ci-dessus. Les résultats chiffrés présentés dans ce document proviennent des notebooks `01_eda_and_cleaning`, `02_preprocessing_encoding` et `03_classification_models`, ainsi que de l'application `app.py`. Aucune technique d'apprentissage profond n'est utilisée : le projet se limite aux algorithmes classiques de classification supervisée.

Résumé

Les plateformes de commerce électronique collectent chaque jour des milliers d'avis clients. Savoir, à partir d'un avis, si la cliente **recommande** l'article permet de mieux piloter le catalogue, de détecter les produits décevants et d'orienter le service client. Ce rapport traite cette question comme un problème de **classification binaire supervisée** sur le jeu de données public *Women's E-Commerce Clothing Reviews*, composé de 23 486 avis bruts (23 465 après nettoyage) décrits par 10 attributs : âge, note, titre et texte de l'avis, nombre de retours positifs, ainsi que trois niveaux de catégorisation du produit.

Nous réalisons un audit de qualité des données (21 doublons exacts, 16,22 % de titres manquants, 3,60 % de textes manquants), une analyse exploratoire, puis deux tests d'hypothèses non paramétriques (Mann-Whitney U et χ^2 d'indépendance) avec correction de Benjamini-Hochberg et tailles d'effet. Le prétraitement, encapsulé dans un `ColumnTransformer` ajusté uniquement sur le jeu d'entraînement, combine imputation, standardisation, One-Hot Encoding et TF-IDF (unigrammes et bigrammes) et produit **13 035 variables**. Six modèles sont comparés après optimisation par `GridSearchCV` (scoring F1) : Zero-R (référence), k-NN, Naïve Bayes multinomial, arbre de décision, forêt aléatoire et SVM linéaire.

Le **Linear SVC** obtient les meilleurs résultats sur le jeu de test (4 693 avis) : accuracy 0,9378, F1 0,9622 et ROC-AUC 0,9775, devant la forêt aléatoire (F1 0,9613). Le déséquilibre des classes (82,2 % / 17,8 %) rend l'accuracy seule trompeuse, puisque le modèle de référence atteint déjà 0,8223 ; l'analyse par classe montre que la détection des avis non recommandés reste la difficulté principale (rappel de 0,82 pour le meilleur modèle). Une application Streamlit permet de prédire en direct la recommandation à partir d'un avis saisi et de consulter les performances des modèles.

Mots-clés : data mining ; classification supervisée ; analyse d'avis clients ; TF-IDF ; SVM linéaire ; déséquilibre des classes ; tests d'hypothèses ; Streamlit.

Abstract

This report addresses the prediction of purchase recommendations from women's clothing e-commerce reviews (23,486 raw reviews, 23,465 after cleaning). After a data quality audit, exploratory analysis and two non-parametric hypothesis tests (Mann-Whitney U, chi-square) with Benjamini-Hochberg correction, a leakage-free preprocessing pipeline (imputation, scaling, One-Hot encoding, TF-IDF) yields 13,035 features. Six classical classifiers are tuned with GridSearchCV and compared on a stratified hold-out set of 4,693 reviews. Linear SVC performs best (accuracy 0.9378, F1 0.9622, ROC-AUC 0.9775), while the minority class (non-recommended) remains harder to detect (recall 0.82). A Streamlit application provides live prediction and a dashboard of the model results. No deep learning is used.

Liste des acronymes

Acronyme	Définition
EDA	Exploratory Data Analysis (analyse exploratoire des données)
TF-IDF	Term Frequency – Inverse Document Frequency
OHE	One-Hot Encoding
SVM / SVC	Support Vector Machine / Support Vector Classifier
k-NN	k-Nearest Neighbors (k plus proches voisins)
RF	Random Forest (forêt aléatoire)
DT	Decision Tree (arbre de décision)
NB	Naïve Bayes
ROC-AUC	Aire sous la courbe ROC
FDR	False Discovery Rate (taux de fausses découvertes)
IQR	Interquartile Range (écart interquartile)
CV	Cross-Validation (validation croisée)
NLP	Natural Language Processing (traitement automatique du langage)

Sommaire

Chapitre 1 – Introduction et contexte 1
1.1 Contexte et motivation 1
1.2 Problématique 1
1.3 Questions de recherche 1
1.4 Contributions du projet 2
1.5 Organisation du document 2

Chapitre 2 – Acquisition du jeu de données et audit de qualité 3
2.1 Origine du jeu de données 3
2.2 Vérification du cahier des charges 3
2.3 Dictionnaire des données 3
2.4 Évaluation de la qualité des données 4
2.5 Synthèse du nettoyage 6

Chapitre 3 – Analyse exploratoire des données (EDA) 7
3.1 Démarche 7
3.2 Statistiques descriptives 7
3.3 Distribution de la variable cible 9
3.4 Variables catégorielles 9
3.5 Corrélations entre variables numériques 12
3.6 Synthèse des constats de l'EDA 12

Chapitre 4 – Tests d'hypothèses statistiques 14
4.1 Méthodologie 14
4.2 Hypothèse H1 – Note et recommandation 14
4.3 Hypothèse H2 – Département et recommandation 14
4.4 Tableau récapitulatif 15
4.5 Interprétation et conséquences pour la modélisation 15

Chapitre 5 – Prétraitement et ingénierie des variables 16
5.1 Principe : prétraiter sans fuite de données 16
5.2 Définition de la cible et des prédicteurs 16
5.3 Séparation entraînement / test 16
5.4 Familles de variables et traitements 16
5.5 Construction du préprocesseur 17
5.6 Résultat du prétraitement 18
5.7 Points d'attention méthodologiques 18

Chapitre 6 – Méthodologie de modélisation 19
6.1 Protocole expérimental 19
6.2 Les algorithmes de classification 19
6.3 Espaces de recherche et configurations retenues 21
6.4 Métriques d'évaluation 21

Chapitre 7 – Résultats expérimentaux et comparaison des modèles 22
7.1 Performances globales sur le jeu de test 22
7.2 Analyse par classe 23
7.3 Interprétation 24
7.4 Modèle retenu pour le déploiement 24
7.5 Limites de l'évaluation 24

Chapitre 8 – Application web Streamlit 26
8.1 Objectifs et architecture 26
8.2 Pages de l'application 26
8.3 Conception visuelle 27
8.4 Page de prédiction en direct 27
8.5 Pages d'analyse 29
8.6 Installation et utilisation 31
8.7 Limites de l'application 31

Chapitre 9 – Ingénierie du projet et organisation du travail 32
9.1 Structure du dépôt 32
9.2 Workflow Git 32

Chapitre 10 – Discussion critique, limites et éthique 34
10.1 Synthèse des résultats 34
10.2 Limites de l'étude 34
10.3 Considérations éthiques 34
10.4 Perspectives 35

Chapitre 11 – Conclusion 36
11.1 Réponses aux questions de recherche 36
11.2 Bilan 36

Annexes 37
Annexe A – Espaces de recherche des hyperparamètres 37
Annexe B – Rapport de classification du modèle retenu (Linear SVC) 37
Annexe C – Commandes de reproduction 37
Annexe D – Code pour générer les figures 7.3 et 7.4 37

Bibliographie 39

Liste des figures

Figure 2.1 – Part de valeurs manquantes par variable....4
Figure 2.2 – Boîtes à moustaches des variables Age et Positive Feedback Count....6
Figure 3.1 – Distributions des variables numériques (Age, Rating, Positive Feedback Count)....9
Figure 3.2 – Répartition de la variable cible....9
Figure 3.3 – Répartition des avis par collection, département et catégorie....11
Figure 3.4 – Matrice de corrélation de Pearson....12
Figure 7.1 – Comparaison des six modèles sur le jeu de test....22
Figure 7.2 – Rappel et précision de la classe « non recommandé »....23
Figure 7.3 – Courbes ROC des cinq modèles entraînés....25
Figure 7.4 – Matrices de confusion des modèles....25
Figure 8.1 – Page d'accueil de l'application....27
Figure 8.2 – Page de prédiction : formulaire de saisie....28
Figure 8.3 – Résultat de prédiction : article recommandé....28
Figure 8.4 – Résultat de prédiction : article non recommandé....29
Figure 8.5 – Page « Performance des modèles »....30
Figure 8.6 – Page « Statistiques du dataset »....30
Figure 8.7 – Page « Découvertes clés »....30
Figure 8.8 – Page « Tests d'hypothèses »....31
Figure 9.1 – Page GitHub du dépôt....33

Liste des tableaux

Tableau 2.1 – Dictionnaire des données....3
Tableau 2.2 – Valeurs manquantes par variable (jeu brut)....4
Tableau 2.3 – Valeurs aberrantes détectées par la règle de Tukey....5
Tableau 2.4 – Distribution de la variable cible....6
Tableau 2.5 – Synthèse du nettoyage....6
Tableau 3.1 – Statistiques descriptives des variables quantitatives....7
Tableau 3.2 – Répartition des avis par collection....10
Tableau 3.3 – Répartition des avis par département....10
Tableau 3.4 – Les quinze catégories de produits les plus fréquentes....10
Tableau 3.5 – Matrice de corrélation de Pearson....12
Tableau 4.1 – Synthèse des tests d'hypothèses....15
Tableau 5.1 – Séparation entraînement / test....16
Tableau 5.2 – Familles de variables et traitements....16
Tableau 5.3 – Paramètres des deux vectoriseurs TF-IDF....17
Tableau 5.4 – Dimensions et structure de la matrice après prétraitement....18
Tableau 6.1 – Espaces de recherche et configurations retenues....21
Tableau 7.1 – Performances globales sur le jeu de test....22
Tableau 7.2 – Précision, rappel et F1 par classe....23
Tableau 8.1 – Pages de l'application....26
Tableau 9.1 – Structure du dépôt....32
Tableau A.1 – Rapport de classification du Linear SVC....37

Chapitre 1 – Introduction et contexte

1.1 Contexte et motivation

Le commerce électronique de mode repose largement sur la confiance entre acheteuses : avant de commander, une cliente lit les avis des autres. À l'inverse, les enseignes disposent d'une masse de retours textuels, souvent sous-exploités. Chaque avis contient pourtant trois informations précieuses : une **note** chiffrée, un **texte libre** exprimant le ressenti (coupe, tissu, taille, qualité) et un **indicateur binaire** précisant si la cliente recommande l'article.

Exploiter ces données de manière automatique répond à plusieurs besoins concrets : repérer rapidement les articles dont la perception se dégrade, anticiper les retours, prioriser les réponses du service client et alimenter les systèmes de recommandation. Les méthodes de data mining, et en particulier la classification supervisée, permettent d'apprendre cette relation à partir d'exemples étiquetés.

1.2 Problématique

La question centrale de ce projet est la suivante :

> *À partir de l'avis d'une cliente (texte, titre, note, âge, utilité perçue) et des caractéristiques de l'article (collection, département, catégorie), peut-on prédire de manière fiable si elle **recommandera** l'article ?*

Formellement, soit un vecteur d'entrée x décrivant un avis, on cherche une fonction f telle que f(x) = ŷ ∈ {0, 1}, où ŷ = 1 signifie « article recommandé » et ŷ = 0 « article non recommandé ». La difficulté tient à trois éléments : (i) la nature **textuelle** et hétérogène des données, qui impose une représentation numérique de grande dimension ; (ii) le **déséquilibre** des classes, avec environ 82 % d'avis positifs ; (iii) la présence de **valeurs manquantes** et de doublons dans les données brutes.

1.3 Questions de recherche

- **QR1 – Qualité et déséquilibre.** Quelle est la qualité des données brutes et quelle est la distribution de la variable cible ? Quelles métriques d'évaluation en découlent ?
- **QR2 – Associations statistiques.** La note et le département sont-ils statistiquement associés à la recommandation, et avec quelle force ?
- **QR3 – Correction des tests multiples.** Les conclusions résistent-elles à la correction de Benjamini-Hochberg ?
- **QR4 – Performance des modèles.** Parmi Zero-R, k-NN, Naïve Bayes, arbre de décision, forêt aléatoire et SVM linéaire, lequel généralise le mieux sur des données jamais vues ?
- **QR5 – Classe minoritaire.** Dans quelle mesure les modèles détectent-ils les avis non recommandés, plus rares mais plus utiles pour l'enseigne ?

- **QR6 – Mise en production.** Comment rendre le meilleur modèle utilisable par un utilisateur non spécialiste ?

1.4 Contributions du projet

1. **Audit de qualité** complet du jeu de données brut (volumétrie, doublons, valeurs manquantes, valeurs aberrantes) et nettoyage traçable.
2. **Analyse exploratoire** univariée, bivariée et par catégories, avec matrice de corrélation.
3. **Deux tests d'hypothèses** non paramétriques avec correction FDR et tailles d'effet.
4. **Pipeline de prétraitement sans fuite de données** (`ColumnTransformer`) combinant variables numériques, catégorielles et textuelles.
5. **Benchmark de six modèles** avec optimisation d'hyperparamètres et analyse par classe.
6. **Application Streamlit** de prédiction en direct et de consultation des résultats (performances, statistiques, tests).
7. **Organisation Git** en équipe avec branches de fonctionnalités et documentation du dépôt.

1.5 Organisation du document

Le rapport suit la démarche classique du data mining (CRISP-DM) : compréhension des données (chapitres 2 à 4), préparation (chapitre 5), modélisation (chapitres 6 et 7), déploiement (chapitre 8), puis organisation du travail, discussion et conclusion (chapitres 9 à 11).

Chapitre 2 – Acquisition du jeu de données et audit de qualité

2.1 Origine du jeu de données

Le jeu de données utilisé est *Women's E-Commerce Clothing Reviews*, publié sur Kaggle. Il rassemble des avis réels (anonymisés) laissés par des clientes d'une enseigne de vêtements pour femmes. Dans le projet, le fichier brut est conservé tel quel dans `data/raw/` et n'est jamais modifié : toutes les transformations produisent un nouveau fichier, `data/processed/reviews_clean.csv`. Cette séparation garantit la reproductibilité de l'analyse.

2.2 Vérification du cahier des charges

Le cahier des charges exige au minimum 10 000 observations et 10 attributs. Le fichier brut contient **23 486 lignes** et **11 colonnes**, dont une colonne d'index inutile (`Unnamed: 0`) supprimée dès le chargement. Les deux conditions sont donc satisfaites avec 10 attributs exploitables.

2.3 Dictionnaire des données

Tableau 2.1 – Dictionnaire des données

Variable	Type	Famille	Rôle	Description
Clothing ID	Entier	Identifiant	Exclue	Identifiant du produit (1 206 valeurs distinctes), sans signification numérique
Age	Entier	Quantitative	Prédicteur	Âge de la cliente (18 à 99 ans, 77 valeurs distinctes)
Title	Texte	Textuelle	Prédicteur	Titre de l'avis (13 993 valeurs distinctes)
Review Text	Texte	Textuelle	Prédicteur	Texte libre de l'avis (22 634 valeurs distinctes)
Rating	Entier	Quantitative ordinale	Prédicteur	Note de 1 à 5
Recommended IND	Entier	Binaire	**Cible**	1 = recommande, 0 = ne recommande pas
Positive Feedback Count	Entier	Quantitative	Prédicteur	Nombre de lecteurs ayant jugé l'avis utile (0 à 122)
Division Name	Catégorielle	Catégorielle	Prédicteur	Collection : General, General Petite, Initmates
Department Name	Catégorielle	Catégorielle	Prédicteur	Département : 6 modalités
Class Name	Catégorielle	Catégorielle	Prédicteur	Catégorie de produit : 20 modalités

La modalité « Initmates » est l'orthographe d'origine du jeu de données ; elle est conservée telle quelle pour rester cohérente avec les données.

L'identifiant `Clothing ID` est retiré des prédicteurs : le produit 1 000 n'est pas « plus grand » que le produit 500, et traiter cet identifiant comme une quantité fausserait les modèles.

2.4 Évaluation de la qualité des données

2.4.1 Valeurs manquantes

Tableau 2.2 – Valeurs manquantes par variable (jeu brut)

Variable	Valeurs manquantes	Pourcentage
Title	3 810	16,22 %
Review Text	845	3,60 %
Division Name	14	0,06 %
Department Name	14	0,06 %
Class Name	14	0,06 %
Clothing ID, Age, Rating, Recommended IND, Positive Feedback Count	0	0,00 %

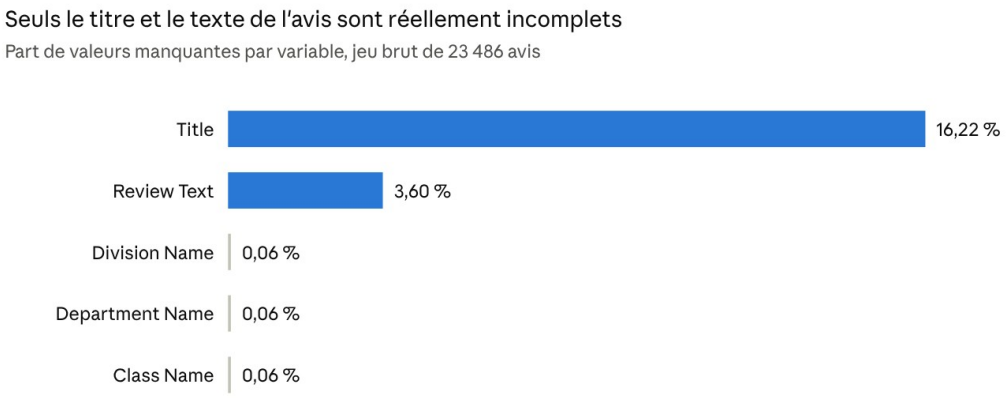


Figure 2.1 – Part de valeurs manquantes par variable

Les titres et textes absents ne rendent pas l'observation invalide : la note et la cible sont renseignées. Elles sont donc conservées et le texte manquant est remplacé par une chaîne vide. Les trois variables catégorielles absentes simultanément pour 14 lignes reçoivent la modalité `Unknown`. Aucune ligne n'est supprimée pour valeur manquante ; la cible n'a aucune valeur absente.

2.4.2 Doublons

Le test de doublons exacts révèle **21 lignes dupliquées**. L'examen montre qu'il s'agit presque exclusivement d'avis à 5 étoiles recommandés, sans titre ni texte, qui ne se distinguent que par l'âge et la catégorie : ils n'apportent aucune information supplémentaire. Ils sont supprimés, ce qui ramène le jeu à **23 465 lignes**.

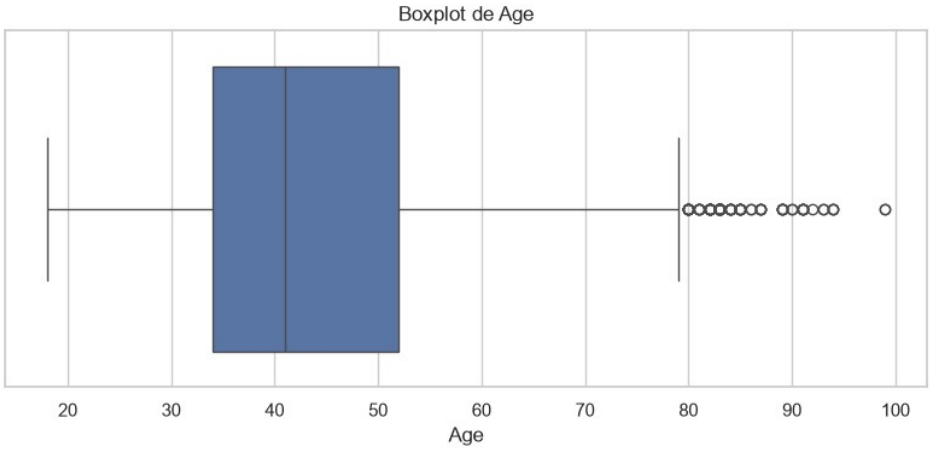
2.4.3 Valeurs aberrantes

La règle de Tukey ($x < Q1 - 1{,}5 \cdot IQR$ ou $x > Q3 + 1{,}5 \cdot IQR$) est appliquée aux variables quantitatives continues :

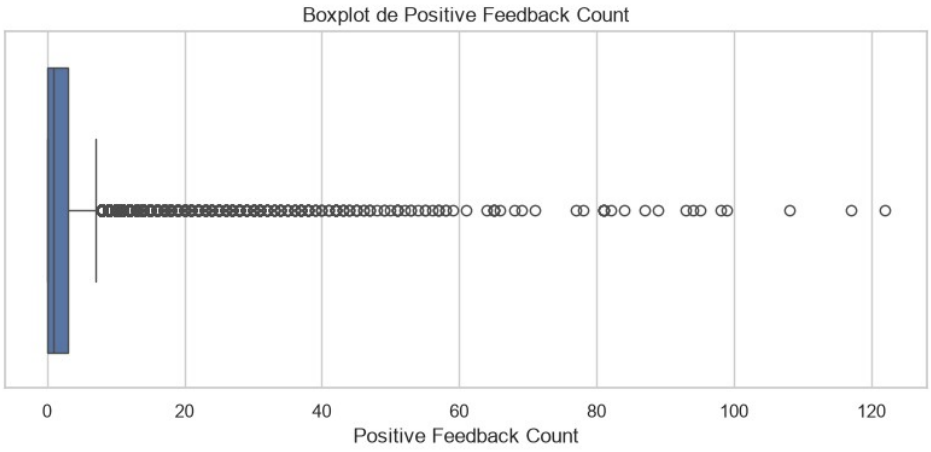
Tableau 2.3 – Valeurs aberrantes détectées par la règle de Tukey

Variable	Q1	Q3	IQR	Borne basse	Borne haute	Outliers	Part
Age	34	52	18	7	79	109	0,46 %
Positive Feedback Count	0	3	3	−4,5	7,5	2 147	9,14 %

Les âges extrêmes (jusqu'à 99 ans) sont plausibles. Les avis très appréciés par les lecteurs (jusqu'à 122 votes « utile ») traduisent un comportement réel et non une erreur de mesure. Ces observations sont donc **conservées** ; l'utilisation de la standardisation et de modèles robustes limite leur influence.



(a) Age



(b) Positive Feedback Count

Figure 2.2 – Boîtes à moustaches des variables Age et Positive Feedback Count

2.4.4 Distribution de la variable cible

Tableau 2.4 – Distribution de la variable cible

Classe	Signification	Effectif	Proportion
0	Non recommandé	4 172	17,76 %
1	Recommandé	19 314	82,24 %

Le rapport de déséquilibre est d'environ 4,6 contre 1. Un classifieur naïf qui prédit toujours « recommandé » atteindrait donc 82 % d'accuracy sans rien apprendre. Pour cette raison, l'évaluation s'appuie sur le **F1-score**, le **ROC-AUC** et les métriques par classe, en plus de l'accuracy.

2.5 Synthèse du nettoyage

Tableau 2.5 – Synthèse du nettoyage

Étape	Lignes
Jeu brut	23 486
Après suppression des doublons	23 465
Après traitement des valeurs manquantes	23 465

À l'issue du nettoyage, le jeu ne contient plus ni doublon ni valeur manquante. Les effectifs des tests statistiques du chapitre 4 diffèrent légèrement (23 486 pour le test de Mann-Whitney, réalisé sur les données brutes, et 23 472 pour le χ^2, qui exclut les 14 lignes sans département) ; cette différence s'explique par le moment où chaque test a été exécuté dans le notebook d'exploration, avant le nettoyage.

Chapitre 3 – Analyse exploratoire des données (EDA)

3.1 Démarche

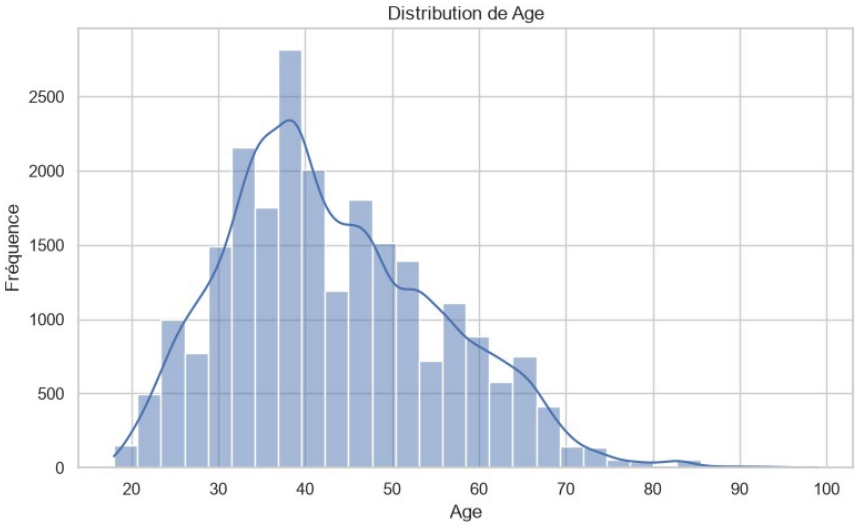
L'analyse exploratoire constitue la phase de diagnostic du data mining : elle révèle la forme des distributions, les déséquilibres, les relations entre variables et les anomalies éventuelles avant toute modélisation. Elle a été menée dans le notebook `01_eda_and_cleaning` à l'aide de pandas, matplotlib et seaborn, sur le jeu brut de 23 486 observations (les statistiques de cette section portent donc sur les données avant suppression des doublons).

3.2 Statistiques descriptives

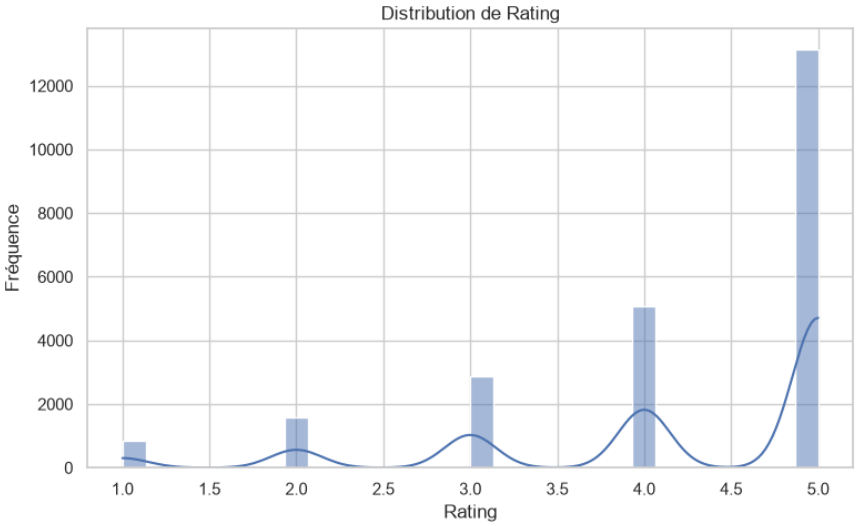
Tableau 3.1 – Statistiques descriptives des variables quantitatives

Variable	Moyenne	Écart-type	Min	Q1	Médiane	Q3	Max
Age	43,20	12,28	18	34	41	52	99
Rating	4,196	1,110	1	4	5	5	5
Positive Feedback Count	2,536	5,702	0	0	1	3	122
Recommended IND	0,822	0,382	0	1	1	1	1

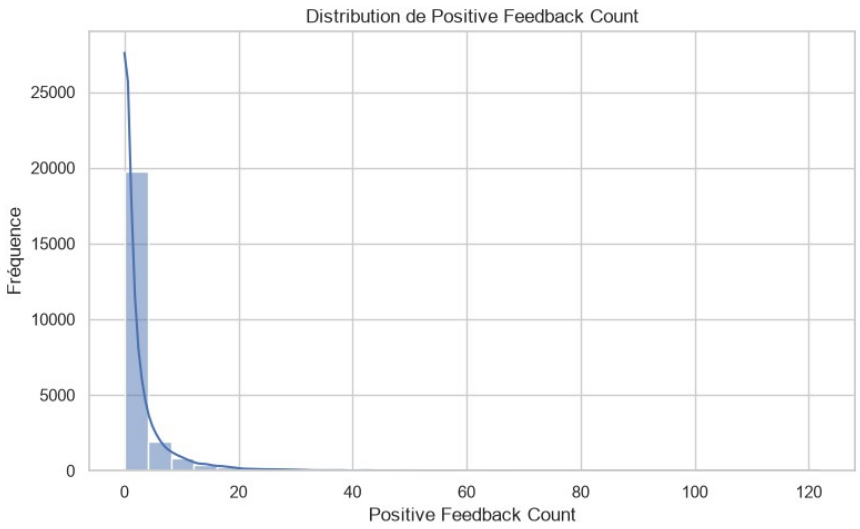
Plusieurs enseignements se dégagent. L'âge moyen (43,2 ans) est supérieur à la médiane (41 ans) : la distribution est légèrement étalée vers la droite, avec une clientèle centrale entre 34 et 52 ans. La note, au contraire, a une moyenne (4,20) inférieure à sa médiane (5) : la distribution est fortement asymétrique à gauche, la majorité des avis étant très positifs. Enfin, le nombre de retours positifs est extrêmement asymétrique : la moitié des avis recueillent au plus un vote « utile », alors que le maximum atteint 122.



(a) Age



(b) Rating



(c) Positive Feedback Count

Figure 3.1 – Distributions des variables numériques (Age, Rating, Positive Feedback Count)

3.3 Distribution de la variable cible

La cible `Recommended IND` compte 19 314 avis recommandés (82,24 %) contre 4 172 non recommandés (17,76 %). Ce déséquilibre justifie le choix des métriques et l'usage de la stratification lors de la séparation train/test.

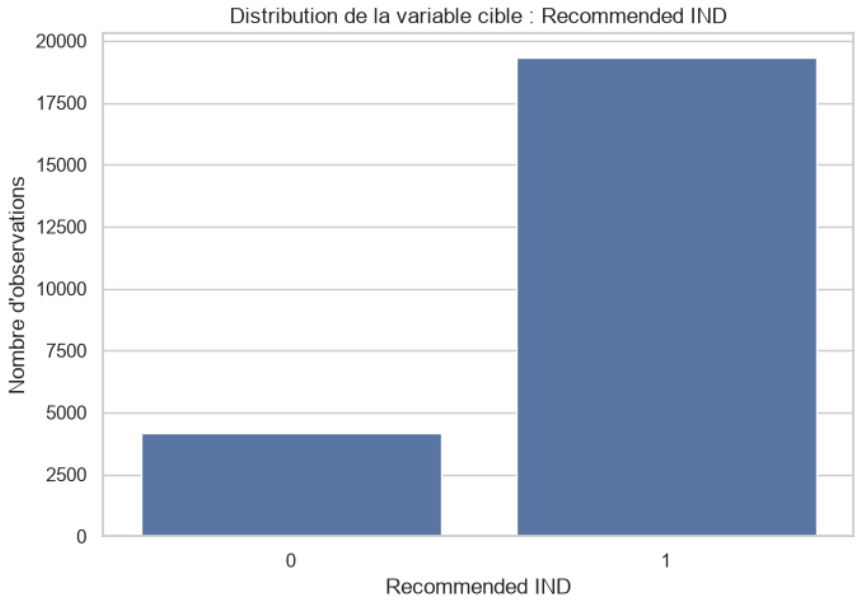


Figure 3.2 – Répartition de la variable cible

3.4 Variables catégorielles

Collection (Division Name)

Tableau 3.2 – Répartition des avis par collection

Modalité	Avis	Part
General	13 850	59,0 %
General Petite	8 120	34,6 %
Initmates	1 502	6,4 %
Manquant	14	0,06 %

Département (Department Name)

Tableau 3.3 – Répartition des avis par département

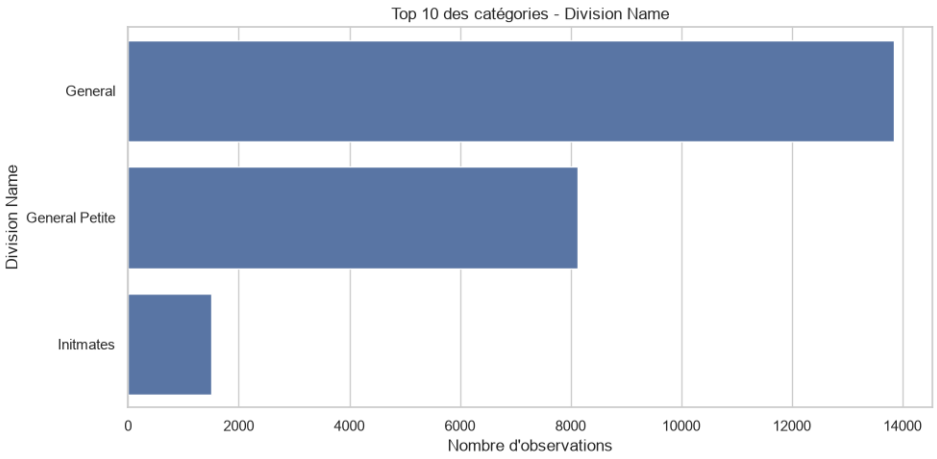
Modalité	Avis	Part
Tops	10 468	44,6 %
Dresses	6 319	26,9 %
Bottoms	3 799	16,2 %
Intimate	1 735	7,4 %
Jackets	1 032	4,4 %
Trend	119	0,5 %
Manquant	14	0,06 %

Catégorie de produit (Class Name) : 20 modalités, dont les quinze plus fréquentes sont :

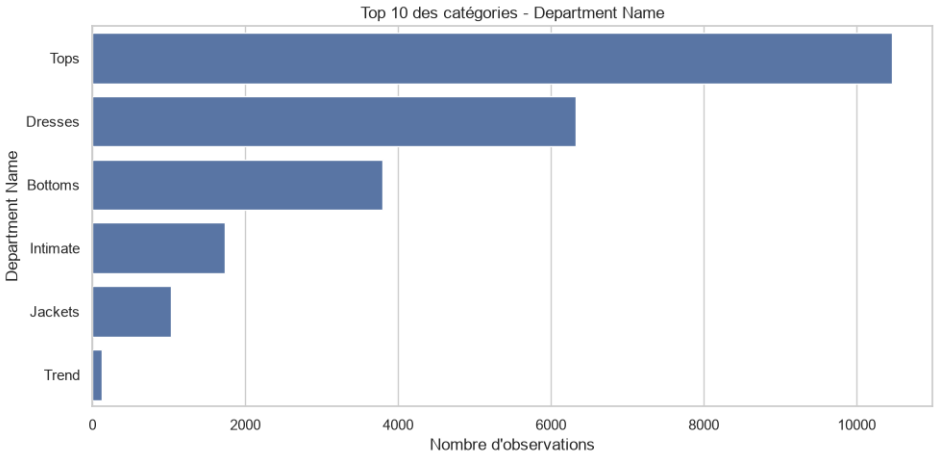
Tableau 3.4 – Les quinze catégories de produits les plus fréquentes

Catégorie	Avis	Catégorie	Avis
Dresses	6 319	Skirts	945
Knits	4 843	Jackets	704
Blouses	3 097	Lounge	691
Sweaters	1 428	Swim	350
Pants	1 388	Outerwear	328
Jeans	1 147	Shorts	317
Fine gauge	1 100	Sleep	228
		Legwear	165

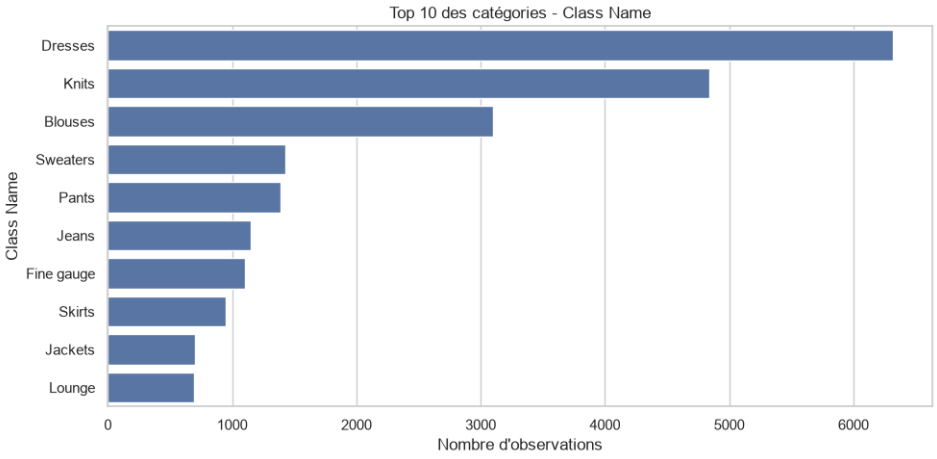
Les cinq modalités restantes sont très rares : elles totalisent 422 avis, soit moins de 2 % du jeu. Trois constats s'imposent : les hauts et les robes représentent plus de 70 % des avis ; la catégorie « Trend » est marginale (119 avis), ce qui rend toute conclusion sur elle fragile ; et la hiérarchie Division > Département > Catégorie introduit une forte redondance entre variables (par exemple, la catégorie « Dresses » correspond exactement au département « Dresses »).



(a) Collection



(b) Département



(c) Catégories de produit

Figure 3.3 – Répartition des avis par collection, département et catégorie

3.5 Corrélations entre variables numériques

La matrice de corrélation de Pearson, incluant la cible binaire (corrélation point-bisériale), est la suivante :

Tableau 3.5 – Matrice de corrélation de Pearson

	Age	Rating	Positive Feedback Count	Recommended IND
Age	1,000	0,027	0,043	0,031
Rating	0,027	1,000	−0,065	**0,792**
Positive Feedback Count	0,043	−0,065	1,000	−0,069
Recommended IND	0,031	0,792	−0,069	1,000

La note est de loin la variable la plus liée à la recommandation (r = 0,79) : une cliente qui attribue 4 ou 5 étoiles recommande presque toujours l'article. L'âge (r = 0,03) et le nombre de retours positifs (r = −0,07) sont très faiblement corrélés à la cible. Aucune paire de prédicteurs n'est fortement corrélée entre elles, ce qui exclut un problème de multicolinéarité parmi les variables numériques.

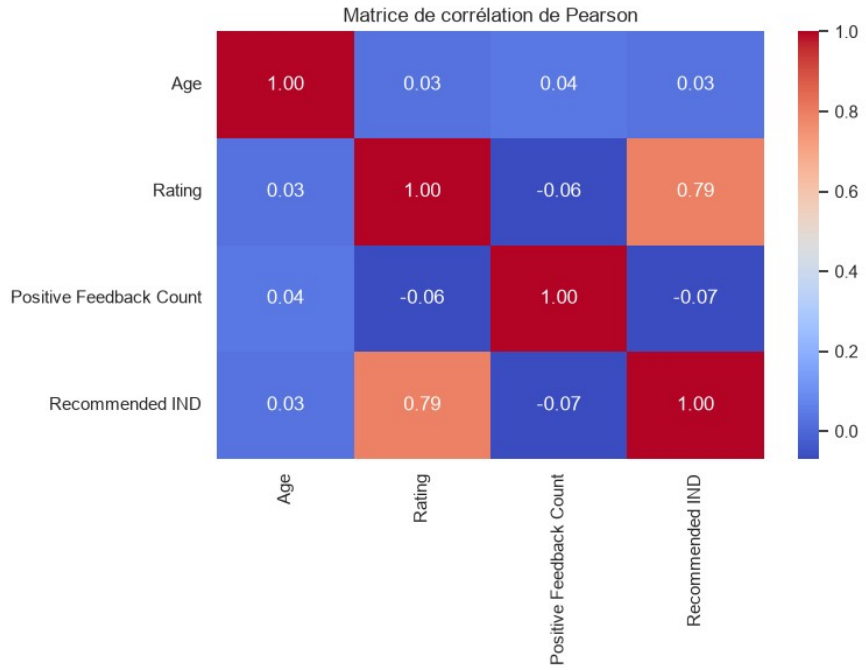


Figure 3.4 – Matrice de corrélation de Pearson

3.6 Synthèse des constats de l'EDA

1. **Classes déséquilibrées** : 82,2 % contre 17,8 %, ce qui impose des métriques adaptées.
2. **Note très informative** : la corrélation de 0,79 annonce un rôle dominant de `Rating` dans les modèles.
3. **Texte partiellement absent** : 16,2 % des titres et 3,6 % des textes manquent, mais ces avis restent exploitables grâce à la note et aux catégories.

4. **Hiérarchie de catégories redondante** : prévoir un encodage tolérant aux modalités rares ou inconnues.
5. **Données quantitatives asymétriques** : la standardisation est utile pour les modèles à distance (k-NN, SVM).

Chapitre 4 – Tests d'hypothèses statistiques

4.1 Méthodologie

Pour vérifier que les différences observées ne relèvent pas du simple hasard d'échantillonnage, deux tests ont été réalisés au seuil $\alpha = 0{,}05$. Les deux variables de l'étude sont de nature ordinale (la note) ou nominale (le département) : des tests **non paramétriques** sont donc appropriés, car ils ne supposent aucune loi normale.

Comme réaliser plusieurs tests simultanément augmente le risque de fausse découverte, nous appliquons la correction de **Benjamini-Hochberg** (FDR). Les p-valeurs brutes sont triées $p(1) \le p(2) \le \dots \le p(m)$, puis ajustées :

$$p_FDR(i) = \min(m \cdot p(i) / i , 1)$$

Avec de grands échantillons, un test devient significatif même pour un effet minuscule. Chaque test est donc accompagné d'une **taille d'effet**, qui mesure l'importance pratique du résultat.

4.2 Hypothèse H1 – Note et recommandation

- **Question** : la distribution de la note diffère-t-elle entre les clientes qui recommandent et celles qui ne recommandent pas ?
- **H0** : les distributions de `Rating` sont identiques dans les deux groupes.
- **H1** : elles diffèrent.
- **Test** : Mann-Whitney U (comparaison de deux groupes indépendants sur une variable ordinale).
- **Effectifs** : 19 314 clientes recommandent, 4 172 ne recommandent pas.
- **Résultats** : U = 78 344 145 ; p-valeur inférieure au seuil de précision numérique (affichée 0) ; médiane de la note égale à **5** chez les recommandantes contre **2** chez les autres.
- **Taille d'effet** : corrélation bisériale par rangs $r = 2U/(n_1 n_0) - 1 \approx$ **0,94**, soit un effet très fort. De façon équivalente, la probabilité qu'une cliente qui recommande ait donné une meilleure note qu'une cliente qui ne recommande pas est d'environ 97 %.
- **Décision** : rejet de H0.

4.3 Hypothèse H2 – Département et recommandation

- **Question** : le département du produit est-il associé à la recommandation ?
- **H0** : `Department Name` et `Recommended IND` sont indépendantes.
- **H1** : elles sont dépendantes.
- **Test** : χ^2 d'indépendance sur le tableau de contingence 6 × 2.

- **Résultats** : $\chi^2 = 50{,}24$; degrés de liberté = $(6-1)(2-1) = 5$; $p = 1{,}24 \times 10^{-9}$; $N = 23\,472$.
- **Taille d'effet** : V de Cramér = $\sqrt{(\chi^2 / N)} \approx$ **0,046**, soit une association très faible.
- **Décision** : rejet de H0 au sens statistique, mais l'effet est négligeable en pratique.

4.4 Tableau récapitulatif

Tableau 4.1 – Synthèse des tests d'hypothèses

ID	Test	Variables	Statistique	p brute	p FDR	Taille d'effet	Décision
H1	Mann-Whitney U	Rating vs Recommended IND	U = 78 344 145	< 0,0001	< 0,0001	r bisérial = 0,94	Rejet de H0
H2	χ^2 d'indépendance	Department Name vs Recommended IND	$\chi^2 = 50{,}24$ (ddl = 5)	$1{,}2 \times 10^{-9}$	$1{,}2 \times 10^{-9}$	V de Cramér = 0,046	Rejet de H0

Les deux conclusions résistent à la correction de Benjamini-Hochberg (m = 2) : la p-valeur de H1 est nulle à la précision numérique, et celle de H2 reste inchangée après ajustement.

4.5 Interprétation et conséquences pour la modélisation

Le contraste entre les deux tailles d'effet est instructif. La **note** sépare presque parfaitement les deux groupes, alors que le **département** n'explique qu'une part infime de la recommandation, malgré une p-valeur extrêmement faible. Autrement dit, significativité statistique et importance pratique sont deux notions distinctes. Pour la modélisation, cela signifie que la note sera un prédicteur dominant, tandis que les catégories de produits joueront un rôle secondaire. L'enjeu réel du texte de l'avis est alors de **corriger les cas où la note et le sentiment divergent** (une note de 3 accompagnée d'un texte négatif, par exemple), ce que seul le traitement du langage peut apporter.

Chapitre 5 – Prétraitement et ingénierie des variables

5.1 Principe : prétraiter sans fuite de données

Une **fuite de données** survient lorsque des informations du jeu de test influencent l'apprentissage, ce qui rend l'évaluation trop optimiste. Pour l'éviter, toutes les transformations qui apprennent des paramètres (médianes, moyennes, écarts-types, vocabulaire TF-IDF, modalités encodées) sont ajustées **uniquement sur le jeu d'entraînement**, puis appliquées telles quelles au jeu de test. Le notebook `02_preprocessing_encoding` implémente ce principe avec un `ColumnTransformer` de scikit-learn.

5.2 Définition de la cible et des prédicteurs

La cible est `Recommended IND` (binaire, valeurs 0 et 1 vérifiées). L'identifiant `Clothing ID` est retiré, ce qui laisse **8 prédicteurs** : Age, Rating, Positive Feedback Count, Division Name, Department Name, Class Name, Title et Review Text.

5.3 Séparation entraînement / test

Le jeu nettoyé (23 465 avis) est séparé de façon **stratifiée** (`stratify=y`) avec `random_state=42` :

Tableau 5.1 – Séparation entraînement / test

Jeu	Avis	Proportion
Entraînement	18 772	80 %
Test	4 693	20 %

La stratification conserve les proportions des classes dans les deux jeux ; le jeu de test contient ainsi 834 avis non recommandés et 3 859 avis recommandés. Le jeu de test n'est utilisé qu'une seule fois, pour l'évaluation finale.

5.4 Familles de variables et traitements

Tableau 5.2 – Familles de variables et traitements

Famille	Variables	Traitement
Numériques (3)	Age, Rating, Positive Feedback Count	Imputation par la médiane puis standardisation (StandardScaler)
Catégorielles (3)	Division Name, Department Name, Class Name	Imputation par la modalité la plus fréquente puis One-Hot Encoding (`handle_unknown="ignore"`)
Texte (2)	Title, Review Text	Vectorisation TF-IDF

5.4.1 Variables numériques

La standardisation transforme chaque variable en z = (x − μ_train) / σ_train. Elle est nécessaire pour les algorithmes fondés sur des distances ou des marges (k-NN, SVM), afin qu'une variable à grande échelle (comme le nombre de retours positifs) ne domine pas les autres. L'imputation par la médiane, robuste aux valeurs extrêmes, est prévue par sécurité, bien qu'aucune valeur numérique ne soit manquante après nettoyage.

5.4.2 Variables catégorielles

Les algorithmes classiques ne manipulent pas de chaînes de caractères. Le One-Hot Encoding crée une colonne binaire par modalité. Le paramètre `handle_unknown="ignore"` rend le pipeline robuste : une modalité absente de l'entraînement (cas fréquent en production) produit simplement un vecteur nul au lieu d'une erreur. Les modalités encodées sont au nombre de 32 : 4 pour la collection (avec `Unknown`), 7 pour le département et 21 pour la catégorie.

5.4.3 Texte et TF-IDF

Le texte est converti en vecteurs numériques par la pondération **TF-IDF** (*Term Frequency – Inverse Document Frequency*) :

$$tf\text{-}idf(t, d) = tf(t, d) \times idf(t) \quad avec \quad idf(t) = \ln((1 + n) / (1 + df(t))) + 1$$

Où tf(t, d) est la fréquence du terme t dans le document d, n le nombre de documents et df(t) le nombre de documents contenant t. Les mots fréquents dans tous les avis (comme « dress ») reçoivent un poids faible, tandis que les mots caractéristiques d'un petit nombre d'avis (« itchy », « disappointing ») reçoivent un poids élevé. Les vecteurs sont ensuite normalisés (norme L2).

Deux vectoriseurs indépendants sont construits :

Tableau 5.3 – Paramètres des deux vectoriseurs TF-IDF

Paramètre	TF-IDF du titre	TF-IDF du texte
Mise en minuscules	Oui	Oui
Mots vides	Anglais	Anglais
N-grammes	(1, 2)	(1, 2)
Taille maximale du vocabulaire	3 000	10 000

L'ajout des **bigrammes** permet de capturer des expressions de deux mots (« runs small », « great quality »). Limiter le vocabulaire aux termes les plus fréquents réduit le bruit et le coût de calcul.

5.5 Construction du préprocesseur

Les quatre traitements (numérique, catégoriel, TF-IDF du titre, TF-IDF du texte) sont assemblés dans un seul `ColumnTransformer`. Il est ajusté avec `fit_transform` sur l'entraînement, puis appliqué avec `transform` seul sur le test :

```
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed  = preprocessor.transform(X_test)
```

5.6 Résultat du prétraitement

Tableau 5.4 – Dimensions et structure de la matrice après prétraitement

Élément	Valeur
Dimensions de X_train	18 772 × 13 035
Dimensions de X_test	4 693 × 13 035
Variables numériques	3
Variables issues du One-Hot Encoding	32
Variables TF-IDF du titre	3 000
Variables TF-IDF du texte	10 000
Total	**13 035**
Valeurs non nulles dans X_train	716 736
Taux de remplissage	0,29 %

La matrice obtenue est une **matrice creuse** (*sparse*) : plus de 99,7 % de ses valeurs sont nulles, ce qui est typique d'une représentation textuelle. Elle est donc stockée au format CSR de SciPy, ce qui économise considérablement la mémoire. Le préprocesseur ajusté est sauvegardé dans `models/preprocessor.pkl`, et les matrices dans `data/processed/` au format `.npz`, afin que l'application utilise exactement la même transformation que l'entraînement.

5.7 Points d'attention méthodologiques

- **Mots de négation.** La liste de mots vides anglais de scikit-learn contient des mots de négation (par exemple « not » et « no »). Les supprimer peut effacer l'information d'une phrase comme « not comfortable ». C'est une limite connue, discutée au chapitre 10.
- **Variables non négatives pour Naïve Bayes.** Le modèle multinomial exige des valeurs positives ; or la standardisation produit des valeurs négatives. Les valeurs négatives sont donc ramenées à zéro pour ce modèle seul (`np.clip`), ce qui fait perdre l'information des valeurs inférieures à la moyenne.
- **Rôle de la note.** La note est incluse comme prédicteur, car elle fait partie des informations disponibles au moment de l'avis. Son poids dominant sera analysé au chapitre 7.

Chapitre 6 – Méthodologie de modélisation

6.1 Protocole expérimental

Le protocole est identique pour tous les modèles, afin que la comparaison soit équitable :

- **Données** : les matrices creuses `X_train` (18 772 × 13 035) et `X_test` (4 693 × 13 035) produites par le notebook 02 ;
- **Optimisation** : `GridSearchCV` avec validation croisée stratifiée sur l'entraînement uniquement, et un **scoring F1** (classe positive) ;
- **Évaluation** : une seule fois, sur le jeu de test jamais vu pendant l'apprentissage ;
- **Reproductibilité** : `random_state=42` pour tous les modèles qui en dépendent.

La validation croisée découpe l'entraînement en k plis stratifiés : à tour de rôle, k − 1 plis servent à apprendre et le dernier à valider. Elle est de 3 plis pour le k-NN, l'arbre de décision, la forêt aléatoire et le SVM linéaire, et de 5 plis pour le Naïve Bayes, dont l'entraînement est très rapide.

6.2 Les algorithmes de classification

6.2.1 Zero-R (modèle de référence)

Zero-R prédit toujours la classe majoritaire (ici « recommandé »). Il n'apprend rien : son rôle est de fournir un **plancher de performance**. Un modèle qui ne le dépasse pas nettement n'a aucun intérêt. Il est implémenté avec `DummyClassifier(strategy="most_frequent")`.

6.2.2 k plus proches voisins (k-NN)

Le k-NN classe un nouvel avis selon la classe majoritaire de ses k voisins les plus proches dans l'espace des variables, au sens de la distance euclidienne :

$$d(x_i, x_j) = \sqrt{\left(\sum_k (x_{ik} - x_{jk})^2\right)}$$

C'est une méthode sans apprentissage explicite : tout le calcul a lieu à la prédiction. En très grande dimension (13 035 variables), les distances perdent de leur pouvoir discriminant (malédiction de la dimensionnalité), ce qui laisse prévoir des résultats limités. Hyperparamètres explorés : nombre de voisins (5, 11, 21), pondération (uniforme ou par distance) et algorithme de recherche exhaustif (`brute`), adapté aux matrices creuses.

6.2.3 Naïve Bayes multinomial

Ce classifieur probabiliste applique le théorème de Bayes en supposant les variables indépendantes conditionnellement à la classe :

$$P(Y = c \mid x) \propto P(Y = c) \cdot \Pi_j P(x_j \mid Y = c)$$

La version multinomiale est adaptée aux comptages et aux poids TF-IDF. Le paramètre de lissage α évite les probabilités nulles pour les mots rares (valeurs explorées : 0,01 ; 0,1 ; 0,5 ; 1 ; 2). Comme indiqué au chapitre 5, les valeurs négatives issues de la standardisation sont ramenées à zéro pour ce modèle.

6.2.4 Arbre de décision

L'arbre découpe récursivement l'espace des variables par des tests binaires, en choisissant à chaque nœud la coupure qui réduit le plus l'impureté. Deux critères sont comparés, l'indice de Gini et l'entropie :

$$Gini(S) = 1 - \Sigma\, p_c^2 \qquad Entropie(S) = -\Sigma\, p_c \cdot \log_2(p_c)$$

L'arbre est facile à interpréter mais sujet au surapprentissage s'il est trop profond. Hyperparamètres explorés : profondeur maximale (illimitée, 5, 10, 20), nombre minimal d'échantillons pour un découpage (2, 5, 10) et critère (Gini, entropie).

6.2.5 Forêt aléatoire

La forêt aléatoire agrège B arbres entraînés sur des échantillons bootstrap, chacun utilisant un sous-ensemble aléatoire de variables à chaque découpage. La prédiction est un vote majoritaire :

$$\hat{f}_RF(x) = \text{vote majoritaire de } \{\, T_b(x),\ b = 1 \dots B \,\}$$

Cette moyenne réduit fortement la variance des arbres individuels. Hyperparamètres explorés : nombre d'arbres (100, 200), profondeur maximale (illimitée, 10, 20) et nombre minimal d'échantillons pour un découpage (2, 5).

6.2.6 SVM linéaire (Linear SVC)

Le SVM linéaire cherche l'hyperplan séparateur de **marge maximale** entre les deux classes. Il minimise une fonction objectif qui combine la largeur de la marge et les erreurs de classement :

$$\min \tfrac{1}{2}\|w\|^2 + C \cdot \Sigma_i \max(0,\ 1 - y_i(w \cdot x_i + b)) \qquad \text{(perte « hinge »)}$$

Le paramètre C arbitre entre une marge large (C petit, plus de régularisation) et un classement strict des exemples d'entraînement (C grand). Les SVM linéaires sont particulièrement efficaces sur les données textuelles creuses de grande dimension, car dans ces espaces les classes sont souvent presque linéairement séparables. Hyperparamètres explorés : C (0,1 ; 1 ; 10) et fonction de perte (`hinge` ou `squared_hinge`), avec `max_iter=2000`.

6.3 Espaces de recherche et configurations retenues

Tableau 6.1 – Espaces de recherche et configurations retenues

Modèle	Espace de recherche	Combinaisons	Plis CV	Configuration retenue
Zero-R	stratégie « most_frequent »	1	–	most_frequent
k-NN	n_neighbors ∈ {5, 11, 21} ; weights ∈ {uniform, distance} ; algorithm = brute	6	3	n_neighbors = 21, weights = uniform, brute
Naïve Bayes	alpha ∈ {0,01 ; 0,1 ; 0,5 ; 1 ; 2}	5	5	alpha = 0,1
Arbre de décision	max_depth ∈ {None, 5, 10, 20} ; min_samples_split ∈ {2, 5, 10} ; criterion ∈ {gini, entropy}	24	3	gini, max_depth = 5, min_samples_split = 5
Forêt aléatoire	n_estimators ∈ {100, 200} ; max_depth ∈ {None, 10, 20} ; min_samples_split ∈ {2, 5}	12	3	n_estimators = 200, max_depth = None, min_samples_split = 5
Linear SVC	C ∈ {0,1 ; 1 ; 10} ; loss ∈ {hinge, squared_hinge}	6	3	C = 0,1, loss = squared_hinge

Pour le k-NN, la meilleure valeur (21) se situe à la borne supérieure de la grille ; une grille plus large (par exemple 31 ou 51 voisins) pourrait encore améliorer ce modèle. Il en va de même pour la profondeur 5 de l'arbre, qui est la plus faible valeur finie testée.

6.4 Métriques d'évaluation

À partir de la matrice de confusion (VP, VN, FP, FN), on calcule :

> *Accuracy = (VP + VN) / (VP + VN + FP + FN) Précision = VP / (VP + FP) Rappel = VP / (VP + FN) F1 = 2 · Précision · Rappel / (Précision + Rappel)*

Le **F1-score** rapporté dans ce document est celui de la classe « recommandé » (classe 1), qui sert de critère d'optimisation du `GridSearchCV`. Comme cette classe représente 82 % des données, il est naturellement élevé, même pour le modèle de référence (0,9025). Pour mesurer la qualité sur la classe minoritaire, nous complétons par la précision, le rappel et le F1 **de chaque classe**, ainsi que par le F1 **macro** (moyenne non pondérée des deux classes). Enfin, le **ROC-AUC** mesure la capacité du modèle à classer un avis recommandé au-dessus d'un avis non recommandé, indépendamment du seuil de décision.

Chapitre 7 – Résultats expérimentaux et comparaison des modèles

7.1 Performances globales sur le jeu de test

Le tableau suivant présente les performances des six modèles sur les 4 693 avis du jeu de test, classés par F1 décroissant.

Tableau 7.1 – Performances globales sur le jeu de test

Rang	Modèle	Accuracy	F1 (classe 1)	ROC-AUC
1	**Linear SVC**	**0,9378**	**0,9622**	**0,9775**
2	Forêt aléatoire	0,9356	0,9613	0,9730
3	Arbre de décision	0,9301	0,9563	0,9684
4	k-NN	0,9186	0,9519	0,9642
5	Naïve Bayes multinomial	0,9052	0,9425	0,9531
6	Zero-R (référence)	0,8223	0,9025	non applicable

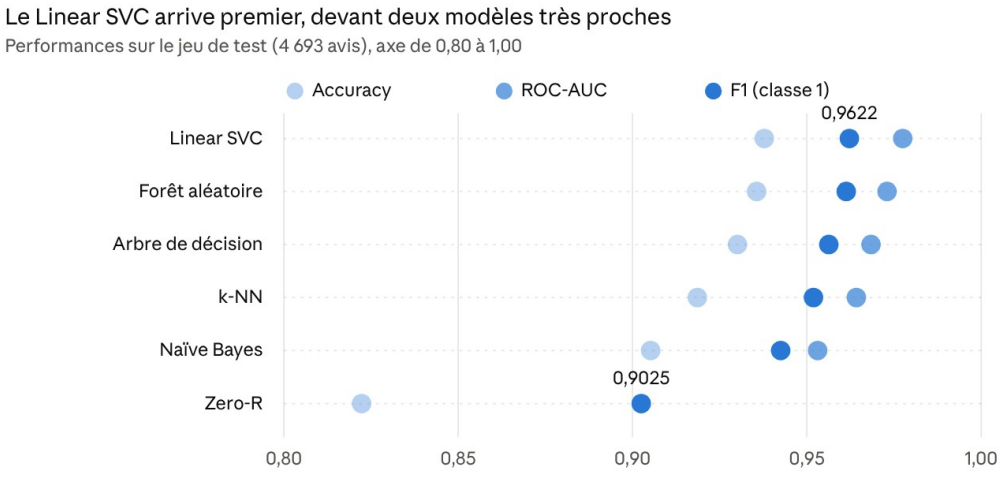


Figure 7.1 – Comparaison des six modèles sur le jeu de test

Le **Linear SVC** arrive premier sur les trois métriques. Tous les modèles entraînés dépassent nettement la référence : le SVM linéaire gagne 11,5 points d'accuracy (0,9378 contre 0,8223) et fait passer le taux d'erreur de 17,8 % à 6,2 %, soit une réduction d'environ 65 %. Les écarts entre les trois meilleurs modèles sont en revanche faibles (0,0009 de F1 entre le SVM et la forêt aléatoire) : ils proviennent d'un seul découpage train/test, sans intervalle de confiance, et ne permettent pas d'affirmer une supériorité statistique du premier sur le deuxième.

7.2 Analyse par classe

Le jeu de test contient 834 avis non recommandés (classe 0) et 3 859 recommandés (classe 1). Les rapports de classification donnent :

Tableau 7.2 – Précision, rappel et F1 par classe

Modèle	Précision classe 0	Rappel classe 0	F1 classe 0	Précision classe 1	Rappel classe 1	F1 classe 1	F1 macro
Linear SVC	0,83	0,82	0,82	0,96	0,96	0,96	0,89
Forêt aléatoire	0,86	0,76	0,81	0,95	0,97	0,96	0,88
Arbre de décision	0,74	**0,93**	0,83	0,98	0,93	0,96	0,89
k-NN	0,87	0,64	0,74	0,93	0,98	0,95	0,84
Naïve Bayes	0,74	0,72	0,73	0,94	0,95	0,94	0,84

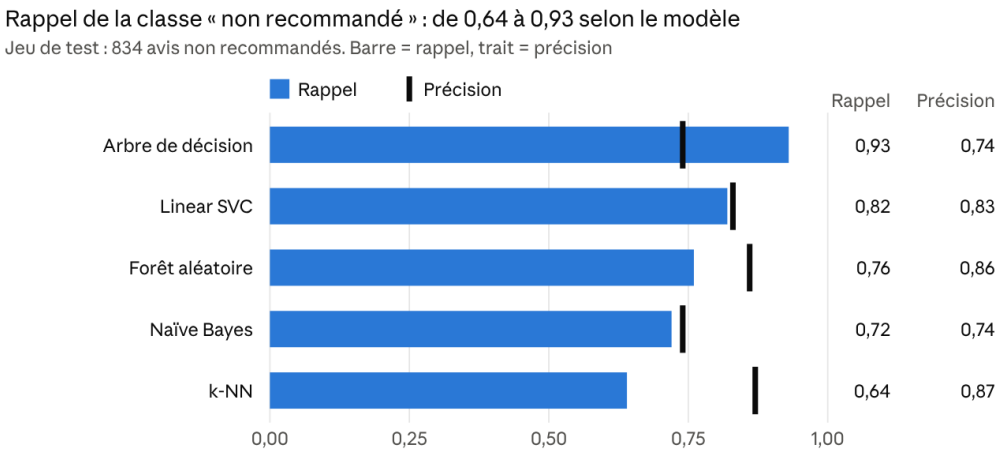


Figure 7.2 – Rappel et précision de la classe « non recommandé »

Cette lecture nuance fortement le classement global :

- **La classe minoritaire est nettement plus difficile.** Pour le meilleur modèle, le F1 passe de 0,96 (classe 1) à 0,82 (classe 0). Avec un rappel de 0,82 sur 834 avis non recommandés, 152 d'entre eux ne sont pas détectés (figure 7.4).
- **Le k-NN manque plus d'un tiers des avis négatifs** (rappel de 0,64) tout en affichant une accuracy de 0,92 : un exemple concret du danger de l'accuracy en cas de déséquilibre.
- **L'arbre de décision détecte le mieux les avis négatifs** (rappel de 0,93), mais au prix de nombreuses fausses alertes (précision de 0,74). Son F1 macro (0,89) égale celui du Linear SVC. Selon l'objectif de l'enseigne (ne manquer aucun client mécontent, ou limiter les fausses alertes), le « meilleur » modèle n'est pas le même.
- **Zero-R** a un rappel nul sur la classe 0 : sa précision pour cette classe est indéfinie, puisqu'il ne la prédit jamais.

7.3 Interprétation

Le bon comportement du SVM linéaire est cohérent avec la nature des données : dans un espace de 13 035 dimensions très creux, des classes souvent presque linéairement séparables favorisent un séparateur linéaire régularisé (C = 0,1 est une valeur faible, donc une forte régularisation). La forêt aléatoire, plus coûteuse, n'apporte pas de gain. Le k-NN souffre de la dimension élevée, et Naïve Bayes de son hypothèse d'indépendance entre mots, peu réaliste pour des n-grammes qui se recouvrent, ainsi que de la perte d'information due au clipping des variables négatives.

Un résultat mérite attention : un arbre limité à une profondeur de 5 niveaux atteint un F1 de 0,9563, très proche des meilleurs modèles. Ceci suggère qu'un nombre restreint de variables, vraisemblablement la **note** (corrélation de 0,79 avec la cible), concentre l'essentiel du signal prédictif. Cette hypothèse, cohérente avec l'analyse exploratoire, mériterait d'être testée par une expérience sans la variable `Rating` (voir les perspectives).

7.4 Modèle retenu pour le déploiement

Le modèle retenu est le **Linear SVC** (C = 0,1, perte squared_hinge), sauvegardé dans `models/best_model.pkl`, accompagné du préprocesseur `models/preprocessor.pkl`. Il offre le meilleur compromis entre performance (meilleur F1 et ROC-AUC), rapidité de prédiction et simplicité. Il présente toutefois une particularité : un SVM linéaire fournit un score de décision et non une probabilité native. L'application affiche donc la classe prédite ; l'obtention de probabilités calibrées (par exemple avec `CalibratedClassifierCV`) fait partie des perspectives du chapitre 10.

7.5 Limites de l'évaluation

- **Un seul découpage.** Les résultats reposent sur un seul jeu de test ; une validation croisée complète ou des intervalles de confiance (par bootstrap) permettraient de départager les modèles proches.
- **Grilles restreintes.** Les espaces de recherche sont volontairement réduits pour limiter le temps de calcul ; certains optimums se trouvent à la borne de la grille.
- **Pas de test sans la note.** L'importance réelle du texte n'est pas isolée.

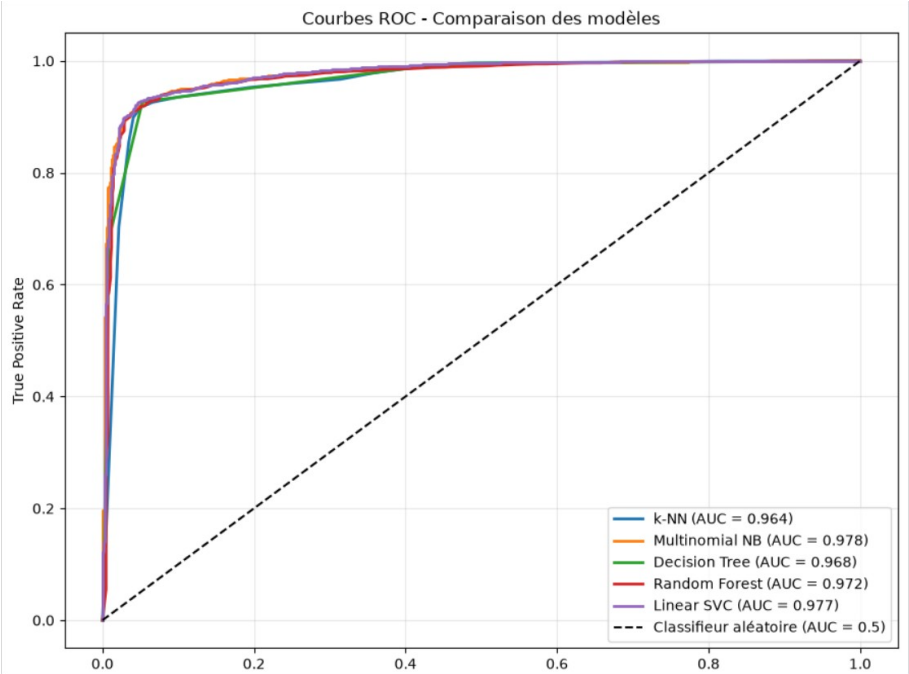


Figure 7.3 – Courbes ROC des cinq modèles entraînés

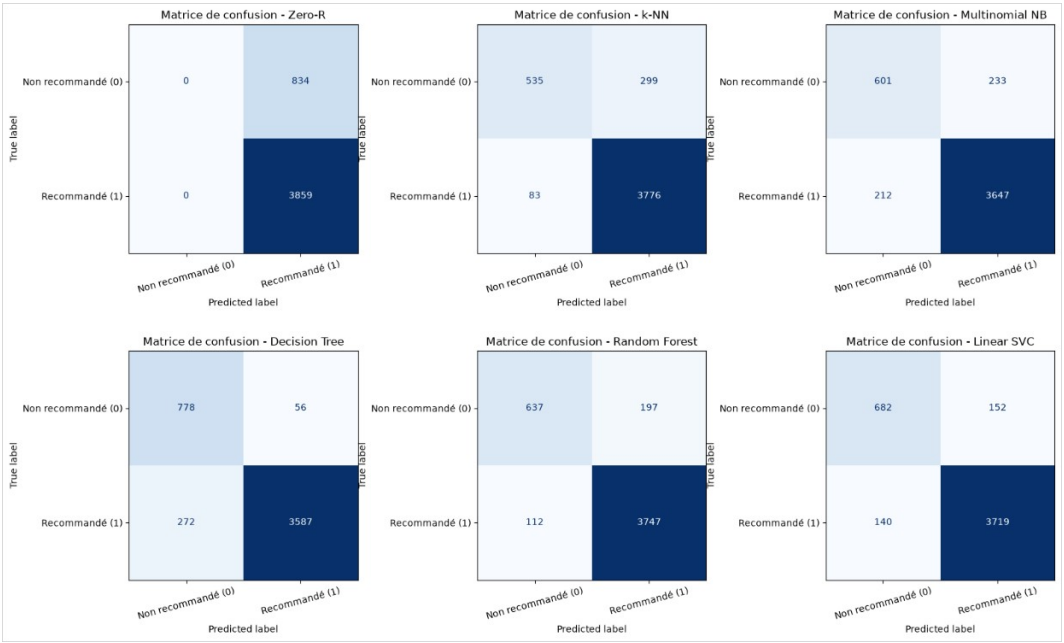


Figure 7.4 – Matrices de confusion des modèles

Chapitre 8 – Application web Streamlit

8.1 Objectifs et architecture

Pour rendre le modèle utilisable sans connaissance en programmation, une application web a été développée avec **Streamlit**, une bibliothèque Python qui transforme un script en tableau de bord interactif. L'application (`src/app.py`) charge au démarrage le modèle `best_model.pkl` et le préprocesseur `preprocessor.pkl` avec `joblib`, en les mettant en cache (`st.cache_resource`) pour qu'ils ne soient chargés qu'une seule fois. Le flux de prédiction est le suivant :

1. l'utilisateur renseigne la fiche produit, le profil de la cliente et son avis dans un formulaire ;
2. les saisies sont assemblées dans un tableau pandas d'une ligne, avec exactement les huit colonnes attendues par le préprocesseur ;
3. `preprocessor.transform` produit le vecteur de 13 035 variables ;
4. `model.predict` renvoie la classe, qui est affichée avec un message explicite.

L'utilisation du même préprocesseur que lors de l'entraînement garantit que les données saisies sont transformées exactement comme les données d'apprentissage.

8.2 Pages de l'application

L'application est organisée en sept pages, accessibles par un menu de navigation latéral.

Tableau 8.1 – Pages de l'application

Page	Contenu
Accueil	Question de recherche, indicateurs clés (23 465 avis, 6 algorithmes, meilleur F1, modèle champion) et pipeline en quatre étapes
Prédiction en direct	Formulaire, carte de résultat, aperçu de l'avis saisi
Performance des modèles	Tableau comparatif et graphique des métriques
Statistiques du dataset	Répartition de la cible et des départements
Découvertes clés	Quatre constats présentés sous forme question / résultat / interprétation
Tests d'hypothèses	Tableau de synthèse des tests, avec correction FDR calculée dans l'application
À propos	Pile technique et avertissement académique

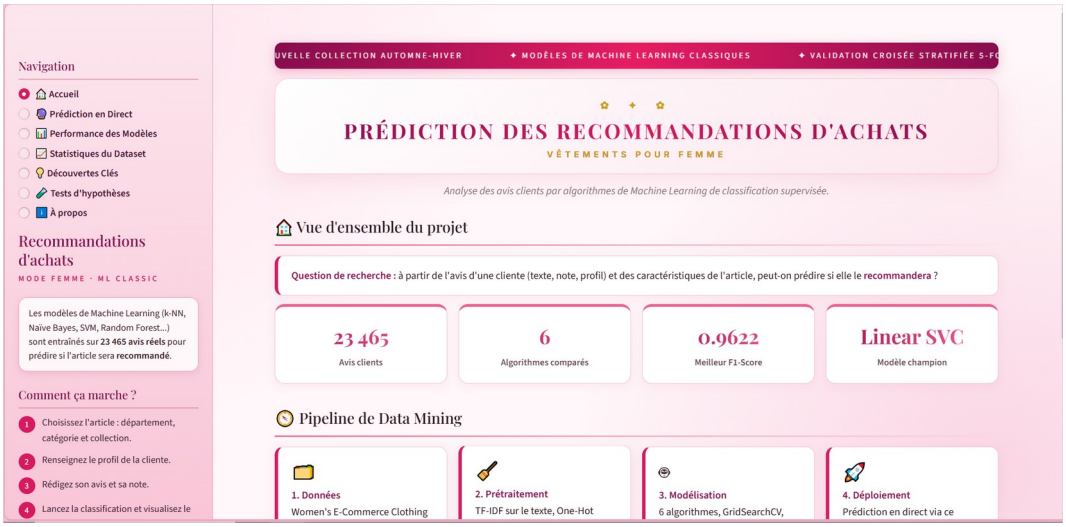


Figure 8.1 – Page d'accueil de l'application

8.3 Conception visuelle

L'interface a été conçue pour être lisible et cohérente avec le thème de la mode féminine. Elle repose sur une palette de roses (rose principal #D81B60, rose clair #F8BBD0, fond #FFF7FA), sur la police Playfair Display pour les titres et la police Inter pour le texte, toutes deux chargées depuis Google Fonts. Une feuille de style CSS, injectée avec `st.markdown`, habille les cartes d'indicateurs, le formulaire, les cartes de résultat, le menu latéral et un bandeau défilant d'en-tête. Le fichier `config.toml` fixe en complément le thème de Streamlit (couleur principale #d498ae, fond #FFF7FA, fond secondaire #FCE4EC, texte #311925).

8.4 Page de prédiction en direct

La page de prédiction est la fonction centrale de l'application. Elle propose des listes déroulantes **dépendantes** : le choix du département filtre les catégories de produit proposées, ce qui empêche les combinaisons incohérentes (une « Robe » ne peut pas appartenir au département « Bas »). Les libellés techniques anglais sont traduits en français pour l'utilisateur, tandis que les valeurs d'origine sont transmises au modèle.

Le profil et l'avis sont saisis dans un formulaire (`st.form`) comprenant l'âge (de 18 à 100 ans), le nombre de votes « utile », la note de 1 à 5 étoiles (curseur à étoiles), le titre et le texte de l'avis. Le formulaire est pré-rempli avec un avis positif en anglais, ce qui permet de tester immédiatement l'application. Trois contrôles protègent son fonctionnement : un message d'erreur est affiché si les fichiers du modèle sont introuvables, un avertissement apparaît si le texte de l'avis est vide, et toute exception pendant la prédiction est capturée et signalée à l'utilisateur.

Le résultat s'affiche dans une carte colorée : verte avec le message « Article recommandé » ou rouge avec « Article non recommandé », accompagnée de la classe prédite. Un aperçu de l'avis saisi, avec sa note en étoiles, est présenté en dessous. Une barre de confiance est prévue lorsque le modèle expose des probabilités (`predict_proba`).

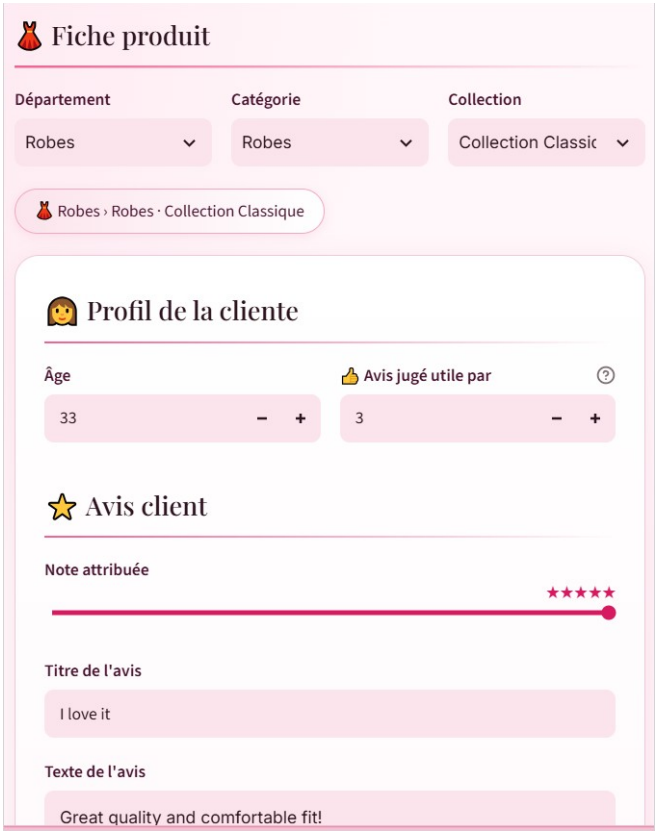


Figure 8.2 – Page de prédiction : formulaire de saisie



Figure 8.3 – Résultat de prédiction : article recommandé



Figure 8.4 – Résultat de prédiction : article non recommandé

8.5 Pages d'analyse

- **Performance des modèles** : tableau comparatif des six modèles (accuracy, F1, ROC-AUC) et graphique en barres des métriques, repris des résultats du notebook 03.
- **Statistiques du dataset** : graphiques en barres de la répartition de la variable cible et des avis par département.
- **Découvertes clés** : quatre constats présentés dans des volets dépliables, chacun avec son résultat et son interprétation.
- **Tests d'hypothèses** : tableau de synthèse des deux tests, avec les p-valeurs ajustées par la méthode de Benjamini-Hochberg, calculées directement dans l'application.
- **À propos** : technologies utilisées, avertissement académique.

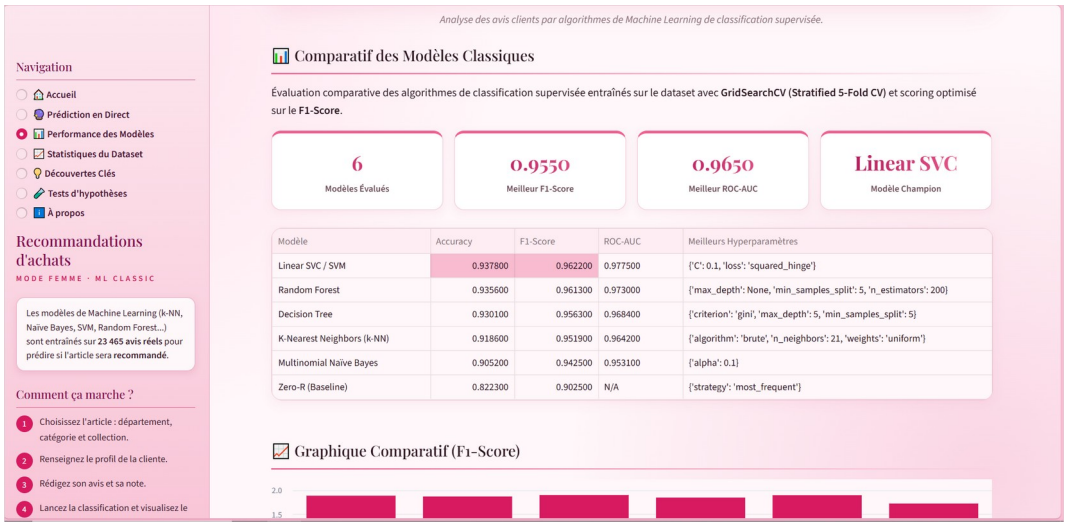


Figure 8.5 – Page « Performance des modèles »

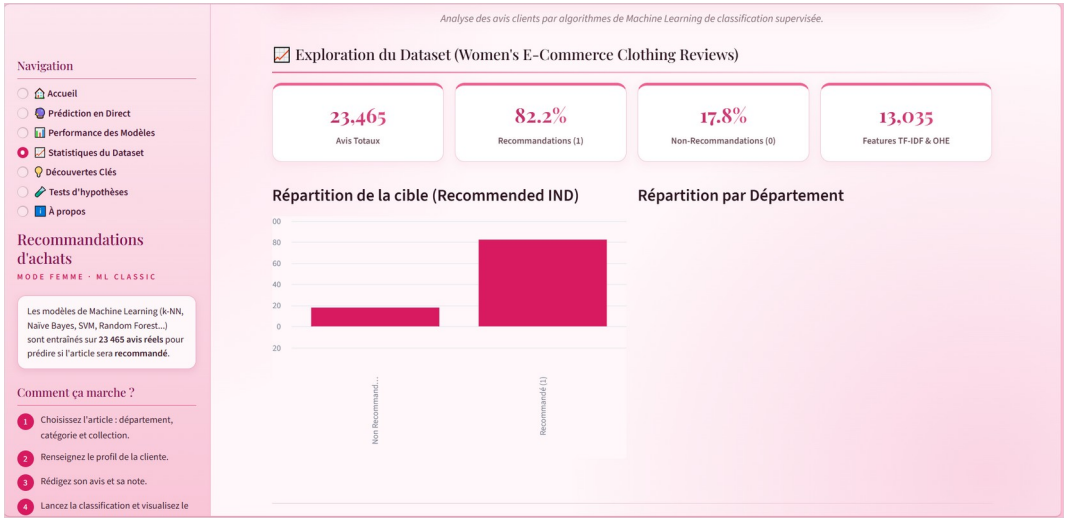


Figure 8.6 – Page « Statistiques du dataset »

[FIGURE 8.7 À INSÉRER]

Capture d'écran de la page « Découvertes clés » (premier volet ouvert).

Figure 8.7 – Page « Découvertes clés »

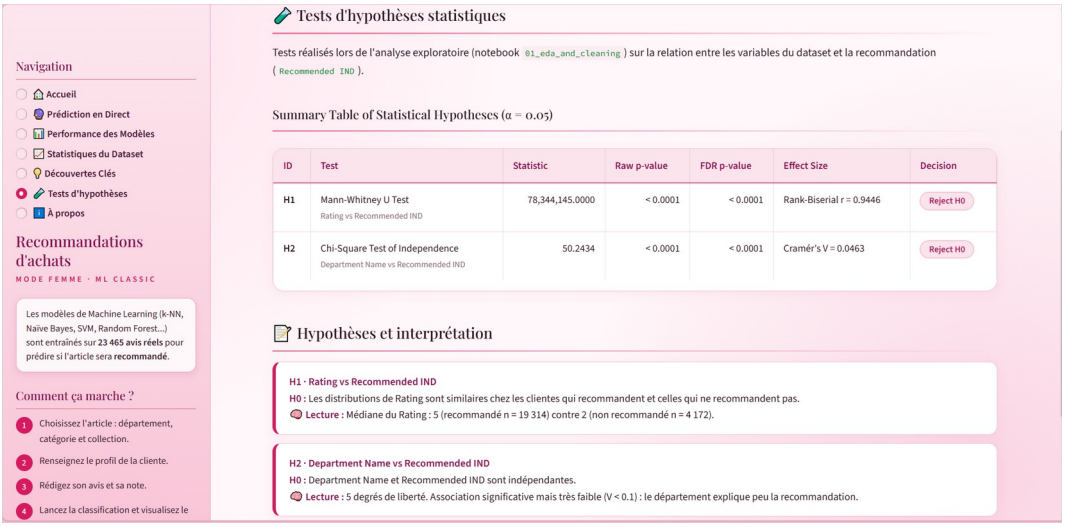


Figure 8.8 – Page « Tests d'hypothèses »

8.6 Installation et utilisation

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python -m streamlit run src/app.py
```

L'application s'ouvre dans le navigateur à l'adresse `http://localhost:8501`. Pour l'utiliser : (1) choisir le département, la catégorie et la collection ; (2) renseigner l'âge, le nombre de votes « utile » et la note ; (3) saisir le titre et le texte de l'avis **en anglais**, langue des données d'entraînement ; (4) cliquer sur « Prédire la recommandation » et lire le résultat.

8.7 Limites de l'application

- Le modèle n'a appris que sur des avis **en anglais** : un avis en français donnera un résultat peu fiable, car presque aucun mot ne figure dans le vocabulaire.
- Le Linear SVC ne fournit pas de probabilités natives : la barre de confiance, prévue pour les modèles qui exposent `predict_proba`, n'est pas affichée avec ce modèle, et l'application indique seulement la classe prédite.
- Les valeurs de performance affichées sont celles du notebook 03, reportées dans le code : elles doivent être mises à jour si les modèles sont réentraînés.
- L'application dépend des fichiers `best_model.pkl` et `preprocessor.pkl` du dossier `models/` ; un message d'erreur est affiché s'ils sont absents.
- Il s'agit d'un outil pédagogique : ses prédictions sont indicatives.

Chapitre 9 – Ingénierie du projet et organisation du travail

9.1 Structure du dépôt

Tableau 9.1 – Structure du dépôt

Dossier / fichier	Contenu
`data/`	Données brutes (`raw/`) et nettoyées (`processed/`)
`notebooks/`	`01_eda_and_cleaning`, `02_preprocessing_encoding`, `03_classification_models`
`models/`	`best_model.pkl`, `preprocessor.pkl`, vectoriseurs TF-IDF
`src/`	Application Streamlit et code source
`docs/`	Rapport du mini-projet
`README.md`	Présentation, installation et utilisation
`requirements.txt`	Dépendances Python
`.gitignore`	Fichiers exclus du suivi (environnement virtuel, caches)

La chaîne de traitement est séquentielle : le notebook 01 produit le jeu nettoyé, le notebook 02 produit le préprocesseur et les matrices, le notebook 03 produit le modèle, et l'application consomme ces artefacts.

9.2 Workflow Git

Le travail en équipe repose sur le principe des **branches de fonctionnalité** : la branche `main` contient la version stable, et chaque évolution est développée sur une branche dédiée avant d'être fusionnée par *pull request* après relecture. Les messages de commit suivent la convention **Conventional Commits** (`feat:`, `fix:`, `docs:`, `refactor:`, `test:`), qui rend l'historique lisible. L'historique du dépôt reflète ce déroulement : mise en place de la structure initiale, intégration successive des modèles (k-NN, Naïve Bayes, arbre de décision), mise à jour des notebooks et de l'application, ajout du rapport et du fichier README.

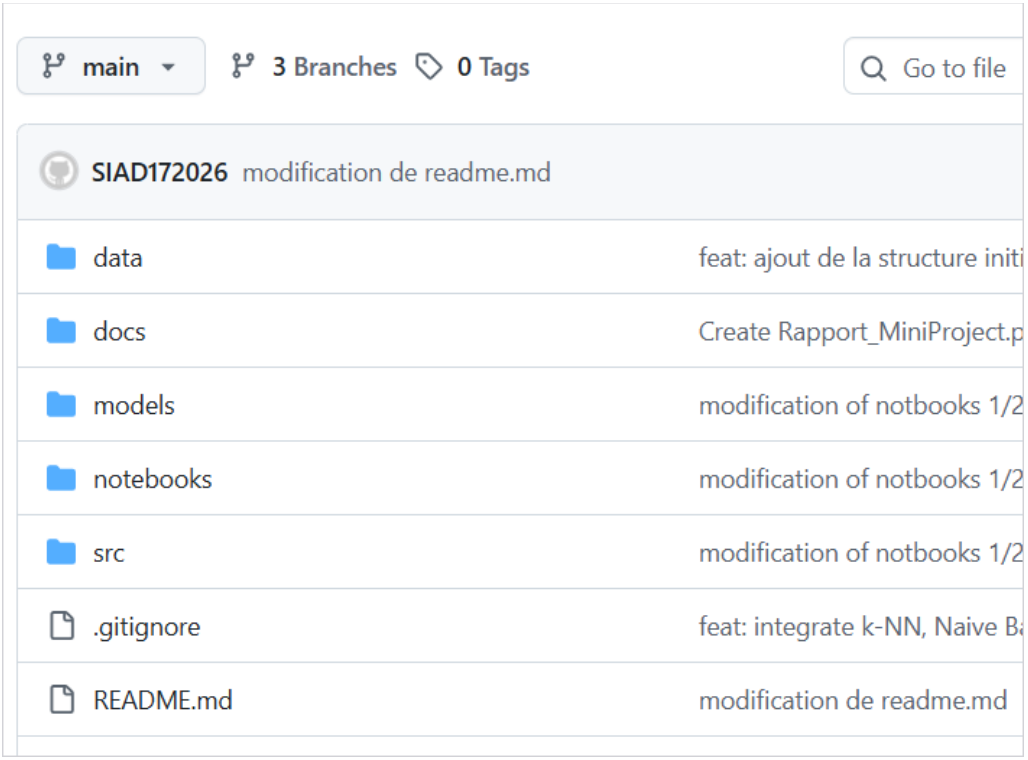


Figure 9.1 – Page GitHub du dépôt

Chapitre 10 – Discussion critique, limites et éthique

10.1 Synthèse des résultats

1. La recommandation est **fortement liée à la note** (corrélation de 0,79, effet bisérial de 0,94), alors que le département n'a qu'une influence marginale (V de Cramér de 0,046).
2. Le **Linear SVC** est le meilleur modèle global (accuracy 0,9378, F1 0,9622, ROC-AUC 0,9775), mais l'écart avec la forêt aléatoire n'est pas démontré statistiquement.
3. L'**accuracy est trompeuse** : le modèle de référence atteint 0,8223. L'analyse par classe est indispensable.
4. La **classe minoritaire** (avis non recommandés) reste la plus difficile à détecter, avec un rappel de 0,82 pour le meilleur modèle et de 0,64 pour le k-NN.
5. Le meilleur modèle selon le critère retenu dépend de l'objectif métier : minimiser les avis négatifs manqués (arbre de décision) ou maximiser l'équilibre général (Linear SVC).

10.2 Limites de l'étude

- **Évaluation sur un seul découpage** : pas d'intervalle de confiance ni de test de comparaison entre modèles (par exemple le test de McNemar).
- **Poids dominant de la note** : une grande partie du signal vient d'une variable numérique simple, ce qui limite l'apport mesuré du texte. L'expérience sans `Rating` n'a pas été réalisée.
- **Traitement du langage rudimentaire** : le TF-IDF ignore l'ordre des mots au-delà des bigrammes, ne gère pas l'ironie, et la suppression des mots vides anglais élimine des négations (« not », « no »).
- **Langue unique** : le modèle ne fonctionne que sur des avis en anglais.
- **Naïve Bayes pénalisé** par le remplacement des valeurs négatives par zéro.
- **Provenance des données** : le jeu provient d'une seule enseigne et reflète sa clientèle ; les conclusions ne se généralisent pas automatiquement à d'autres marchés. Les modalités rares (« Trend », 119 avis) donnent des estimations peu fiables.
- **Étiquette potentiellement bruitée** : un même avis peut être jugé différemment selon les clientes (une note de 3 peut s'accompagner ou non d'une recommandation).

10.3 Considérations éthiques

- **Données personnelles.** Les avis sont anonymisés dans le jeu utilisé, mais un déploiement réel impliquerait le traitement de données à caractère personnel, soumis en Algérie à la loi n° 18-07 relative à la protection des personnes physiques dans le traitement de ces données, et à des règles comparables ailleurs (RGPD).

- **Biais.** L'âge est utilisé comme prédicteur ; il convient de vérifier que les performances sont équivalentes selon les tranches d'âge avant toute utilisation réelle, afin d'éviter de défavoriser un groupe.
- **Usage des prédictions.** Un outil de ce type doit appuyer une décision humaine et non la remplacer, en particulier lorsqu'un avis négatif détecté à tort ou manqué a des conséquences pour le client.
- **Avertissement.** L'application est développée à des fins pédagogiques ; ses prédictions sont indicatives.

10.4 Perspectives

1. **Expérience sans la note** pour isoler la contribution du texte, et mesure de l'importance des variables.
2. **Traitement du déséquilibre** : pondération des classes (`class_weight="balanced"`), ajustement du seuil de décision et suivi du F1 macro, afin d'améliorer le rappel de la classe 0.
3. **Gestion des négations** : conserver les mots « not », « no » et « never » dans le vocabulaire, ou utiliser des n-grammes de caractères.
4. **Nouvelles variables** : longueur de l'avis, nombre de mots, score de sentiment lexical (VADER ou TextBlob).
5. **Techniques de data mining complémentaires** : classification non supervisée des clientes (K-Means), règles d'association (Apriori) entre catégories et notes, modélisation thématique (NMF ou LDA) des avis négatifs pour identifier les causes de mécontentement (taille, tissu, coupe).
6. **Évaluation plus robuste** : validation croisée répétée, intervalles de confiance par bootstrap, test de McNemar entre modèles.
7. **Régression logistique calibrée** comme alternative probabiliste au SVM linéaire, ou calibration de ce dernier avec CalibratedClassifierCV, afin d'afficher une probabilité de recommandation dans l'application.

Chapitre 11 – Conclusion

11.1 Réponses aux questions de recherche

- **QR1 – Qualité et déséquilibre.** Le jeu brut de 23 486 avis contient 21 doublons exacts, 16,22 % de titres et 3,60 % de textes manquants. Les classes sont déséquilibrées (82,2 % / 17,8 %), ce qui impose le recours au F1, au ROC-AUC et aux métriques par classe.
- **QR2 – Associations.** La note (effet bisérial de 0,94) est fortement associée à la recommandation ; le département l'est très faiblement (V de Cramér de 0,046).
- **QR3 – Tests multiples.** Les deux conclusions résistent à la correction de Benjamini-Hochberg.
- **QR4 – Performance.** Le Linear SVC est le meilleur modèle (F1 0,9622, ROC-AUC 0,9775), suivi de près par la forêt aléatoire.
- **QR5 – Classe minoritaire.** Elle reste plus difficile : le rappel est de 0,82 pour le meilleur modèle, de 0,93 pour l'arbre de décision (au prix d'une précision de 0,74) et de 0,64 pour le k-NN.
- **QR6 – Mise en production.** Une application Streamlit permet la prédiction en direct à partir d'un avis saisi, ainsi que la consultation des performances des modèles, des statistiques du jeu de données et des tests.

11.2 Bilan

Ce projet a parcouru l'ensemble de la démarche de data mining : compréhension et nettoyage des données, analyse statistique, préparation d'une représentation numérique du texte, comparaison rigoureuse de six modèles classiques, puis déploiement dans une application interactive. Il montre qu'un modèle linéaire classique, bien préparé, atteint de très bonnes performances sur un problème de classification de texte, sans recours à l'apprentissage profond. Il met aussi en évidence l'importance de la lecture critique des métriques : face à des classes déséquilibrées, un score global élevé peut masquer des faiblesses sur la classe qui compte le plus pour l'utilisateur. Les perspectives identifiées (analyse sans la note, gestion des négations, traitement du déséquilibre, modélisation thématique) tracent des voies concrètes pour prolonger ce travail.

Annexes

Annexe A – Espaces de recherche des hyperparamètres

```
param_grid_knn = {"n_neighbors": [5, 11, 21], "weights": ["uniform", "distance"],
"algorithm": ["brute"]}
param_grid_nb  = {"alpha": [0.01, 0.1, 0.5, 1.0, 2.0]}
param_grid_dt  = {"max_depth": [None, 5, 10, 20], "min_samples_split": [2, 5, 10],
"criterion": ["gini", "entropy"]}
param_grid_rf  = {"n_estimators": [100, 200], "max_depth": [None, 10, 20],
"min_samples_split": [2, 5]}
param_grid_svc = {"C": [0.1, 1.0, 10.0], "loss": ["hinge", "squared_hinge"]}
```

Annexe B – Rapport de classification du modèle retenu (Linear SVC)

Tableau A.1 – Rapport de classification du Linear SVC

Classe	Précision	Rappel	F1	Support
0 (non recommandé)	0,83	0,82	0,82	834
1 (recommandé)	0,96	0,96	0,96	3 859
Accuracy			0,94	4 693
Moyenne macro	0,90	0,89	0,89	4 693
Moyenne pondérée	0,94	0,94	0,94	4 693

Annexe C – Commandes de reproduction

```
git clone https://github.com/RSD018/Prediction-des-recommandations--dachats--de-
vetements--pour--femmes.git
cd Prediction-des-recommandations--dachats--de-vetements--pour--femmes
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
# Exécuter dans l'ordre les notebooks 01, 02 puis 03
python -m streamlit run src/app.py
```

Annexe D – Code pour générer les figures 7.3 et 7.4

À ajouter dans une nouvelle cellule à la fin du notebook `03_classification_models`, puis à exécuter. Les images sont enregistrées dans `docs/figures/`.

```
import matplotlib.pyplot as plt
from sklearn.metrics import RocCurveDisplay, ConfusionMatrixDisplay

FIG_DIR = PROJECT_ROOT / "docs" / "figures"
FIG_DIR.mkdir(parents=True, exist_ok=True)

models = {
    "k-NN": (best_knn, X_test),
    "Naive Bayes": (best_nb, X_te_nb),
    "Decision Tree": (best_dt, X_test),
    "Random Forest": (best_rf, X_test),
    "Linear SVC": (best_linear_svc, X_test),
}

# Figure 7.3 : courbes ROC
fig, ax = plt.subplots(figsize=(7, 6))
for name, (m, X) in models.items():
    RocCurveDisplay.from_estimator(m, X, y_test, name=name, ax=ax)
ax.plot([0, 1], [0, 1], "k--", label="Hasard")
ax.set_title("Courbes ROC des modèles (jeu de test)")
ax.legend(loc="lower right")
fig.savefig(FIG_DIR / "fig_7_3_courbes_roc.png", dpi=200, bbox_inches="tight")
plt.show()

# Figure 7.4 : matrices de confusion
fig, axes = plt.subplots(2, 3, figsize=(14, 8))
for ax, (name, (m, X)) in zip(axes.ravel(), models.items()):
    ConfusionMatrixDisplay.from_estimator(m, X, y_test, ax=ax, cmap="Blues",
colorbar=False)
    ax.set_title(name)
axes.ravel()[-1].axis("off")
fig.tight_layout()
fig.savefig(FIG_DIR / "fig_7_4_matrices_confusion.png", dpi=200, bbox_inches="tight")
plt.show()
```

Bibliographie

1. Benjamini, Y. et Hochberg, Y. (1995). Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B*, 57(1), 289–300.
2. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
3. Breiman, L., Friedman, J., Olshen, R. et Stone, C. (1984). *Classification and Regression Trees*. Wadsworth.
4. Cortes, C. et Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.
5. Cover, T. et Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27.
6. Cramér, H. (1946). *Mathematical Methods of Statistics*. Princeton University Press.
7. Fan, R.-E., Chang, K.-W., Hsieh, C.-J., Wang, X.-R. et Lin, C.-J. (2008). LIBLINEAR: a library for large linear classification. *Journal of Machine Learning Research*, 9, 1871–1874.
8. Han, J., Kamber, M. et Pei, J. (2011). *Data Mining: Concepts and Techniques* (3e éd.). Morgan Kaufmann.
9. Mann, H. B. et Whitney, D. R. (1947). On a test of whether one of two random variables is stochastically larger than the other. *Annals of Mathematical Statistics*, 18(1), 50–60.
10. Pedregosa, F. et al. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
11. Platt, J. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. Dans *Advances in Large Margin Classifiers*, MIT Press, 61–74.
12. Salton, G. et Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing and Management*, 24(5), 513–523.
13. Wirth, R. et Hipp, J. (2000). CRISP-DM: towards a standard process model for data mining. *Actes de PAKDD*.
14. *Women's E-Commerce Clothing Reviews*, jeu de données public, Kaggle. [Compléter l'auteur et la date d'accès.]
15. Documentation Streamlit, docs.streamlit.io. [Compléter la date de consultation.]